

A3: Predicting Car Price III

Student: st127314

Problem: convert cleaned selling prices into four classes, implement multinomial logistic regression and classification metrics from scratch, add optional L2 regularisation, track experiments in MLflow, and deploy the selected model.

1. Reproducible setup

The shared modules below are used by this notebook, the unit tests, and the Dash app. Keeping one implementation avoids a notebook-only class that cannot be unpickled in deployment.

```
In [15]: from pathlib import Path
import json, sys
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import classification_report

ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
sys.path.insert(0, str(ROOT / 'app' / 'code'))
sys.path.insert(0, str(ROOT / 'training'))
from data_prep import load_clean_data, bucket_prices, price_bin_edges
from logistic_regression import LogisticRegression, classification_report_fr
from train import EXPERIMENT_NAME, MODEL_NAME, TRACKING_URI
print(EXPERIMENT_NAME, MODEL_NAME, TRACKING_URI)
```

st127314-a3 st127314-a3-model http://127.0.0.1:5000

2. A1/A2 preprocessing and four price classes

I reuse the A1/A2 rules: remove duplicate listings, test-drive cars, incompatible CNG/LPG mileage rows and an impossible odometer value; parse numeric units; extract brand; and group rare brands. Numeric median imputation includes the A1/A2 missing-value indicators; categorical values use most-frequent imputation and one-hot encoding. The three quartile boundaries are learned from training prices only. The outer test split therefore cannot influence either the bins or model selection.

```
In [16]: from train import load_split
X_train, X_test, y_train, y_test, edges = load_split(ROOT / 'datasets' / 'Ca
pd.DataFrame({'class': range(4), 'train_support': np.bincount(y_train, minle
```

```
Out [16]:
```

	class	train_support	test_support	lower	upper
0	0	1468	372	-inf	250000.0
1	1	1255	324	250000.0	409999.0
2	2	1361	336	409999.0	645000.0
3	3	1360	329	645000.0	inf

3. Metrics from scratch

For each class, TP, FP and FN are counted in a one-vs-rest view. Macro averaging gives every class equal weight. Weighted averaging uses `support / total_support` as the class weight. The `/ 4` printed after the weighted sum in the brief would make the result four times too small and would not match scikit-learn, so it is not applied.

```
In [17]: y_true = np.array([0, 0, 1, 1, 2, 2, 3, 3, 3])
y_pred = np.array([0, 1, 1, 1, 2, 3, 3, 0, 3])
ours = classification_report_from_scratch(y_true, y_pred, labels=[0,1,2,3])
theirs = classification_report(y_true, y_pred, labels=[0,1,2,3], output_dict=True)
comparison = pd.DataFrame({
    'scratch': [ours['accuracy'], ours['macro avg']['f1-score'], ours['weighted avg']['f1-score']],
    'sklearn': [theirs['accuracy'], theirs['macro avg']['f1-score'], theirs['weighted avg']['f1-score']],
    index=['accuracy', 'macro F1', 'weighted F1'])
comparison['absolute difference'] = (comparison.scratch - comparison.sklearn)
comparison
```

```
Out [17]:
```

	scratch	sklearn	absolute difference
accuracy	0.666667	0.666667	0.0
macro F1	0.658333	0.658333	0.0
weighted F1	0.659259	0.659259	0.0

Support is the number of true observations belonging to a class in the evaluated data. It is a count, not a performance score. It supplies the weights in weighted averages.

4. Multinomial and ridge logistic regression

The estimator applies a numerically stable softmax to four linear scores and minimises mean cross-entropy with mini-batch gradient descent. `l2_lambda=0` disables regularisation. A positive value adds `lambda * sum(W**2)` and gradient `2 * lambda * W`; the intercept is not penalised. The complete commented implementation is in `app/code/logistic_regression.py`.

```
In [18]: import inspect
print(inspect.getsource(LogisticRegression))
```

```

class LogisticRegression(ClassifierMixin, BaseEstimator):
    """Softmax regression trained with mini-batch gradient descent.

    ``l2_lambda=0`` gives ordinary multinomial logistic regression. A positive
    value adds ``lambda * sum(weights**2)`` to the mean cross-entropy. The
    intercept row is deliberately not penalised.
    """

    def __init__(
        self,
        learning_rate=0.05,
        num_epochs=600,
        batch_size=256,
        l2_lambda=0.0,
        fit_intercept=True,
        tolerance=1e-7,
        patience=20,
        random_state=42,
        verbose=False,
    ):
        self.learning_rate = learning_rate
        self.num_epochs = num_epochs
        self.batch_size = batch_size
        self.l2_lambda = l2_lambda
        self.fit_intercept = fit_intercept
        self.tolerance = tolerance
        self.patience = patience
        self.random_state = random_state
        self.verbose = verbose

    @staticmethod
    def _softmax(scores):
        shifted = scores - np.max(scores, axis=1, keepdims=True)
        exp_scores = np.exp(shifted)
        return exp_scores / exp_scores.sum(axis=1, keepdims=True)

    def _add_intercept(self, X):
        X = np.asarray(X, dtype=float)
        return np.c_[np.ones(X.shape[0]), X] if self.fit_intercept else X

    def _loss(self, X, y_index):
        probabilities = self._softmax(X @ self.weights_)
        data_loss = -np.mean(np.log(np.clip(probabilities[np.arange(len(y_index)), y_index], 1e-15, 1)))
        regularised = self.weights_[1:] if self.fit_intercept else self.weights_
        return float(data_loss + self.l2_lambda * np.sum(regularised**2))

    def fit(self, X, y):
        X = self._add_intercept(X)
        y = _as_1d(y)
        if X.shape[0] != y.size:
            raise ValueError("X and y must contain the same number of rows")
        if self.l2_lambda < 0:
            raise ValueError("l2_lambda must be non-negative")

```

```

self.classes_, y_index = np.unique(y, return_inverse=True)
if self.classes_.size < 2:
    raise ValueError("at least two classes are required")

rng = np.random.default_rng(self.random_state)
self.weights_ = rng.normal(0, 0.01, size=(X.shape[1], self.classes_.
size))
best_loss, stale_epochs = np.inf, 0
self.loss_history_ = []
eye = np.eye(self.classes_.size)

for epoch in range(self.num_epochs):
    permutation = rng.permutation(X.shape[0])
    for start in range(0, X.shape[0], self.batch_size):
        indices = permutation[start : start + self.batch_size]
        xb, yb = X[indices], y_index[indices]
        error = self._softmax(xb @ self.weights_) - eye[yb]
        gradient = xb.T @ error / len(indices)
        penalty_gradient = 2 * self.l2_lambda * self.weights_
        if self.fit_intercept:
            penalty_gradient[0] = 0
        self.weights_ -= self.learning_rate * (gradient + penalty_gr
adient)

    loss = self._loss(X, y_index)
    self.loss_history_.append(loss)
    improvement = best_loss - loss
    if improvement > self.tolerance:
        best_loss, stale_epochs = loss, 0
    else:
        stale_epochs += 1
    if self.verbose and (epoch % 100 == 0 or epoch == self.num_epoch
s - 1):
        print(f"epoch={epoch:4d} loss={loss:.6f}")
    if stale_epochs >= self.patience:
        break

self.n_features_in_ = X.shape[1] - int(self.fit_intercept)
self.n_iter_ = len(self.loss_history_)
return self

def predict_proba(self, X):
    if not hasattr(self, "weights_"):
        raise RuntimeError("fit must be called before predict_proba")
    return self._softmax(self._add_intercept(X) @ self.weights_)

def predict(self, X):
    return self.classes_[np.argmax(self.predict_proba(X), axis=1)]

def score(self, X, y):
    return accuracy(y, self.predict(X))

# Metric methods keep the assignment's requested API on the model class.
accuracy = staticmethod(accuracy)
precision = staticmethod(precision)

```

```

recall = staticmethod(recall)
f1_score = staticmethod(f1_score)
macro_precision = staticmethod(macro_precision)
macro_recall = staticmethod(macro_recall)
macro_f1 = staticmethod(macro_f1)
weighted_precision = staticmethod(weighted_precision)
weighted_recall = staticmethod(weighted_recall)
weighted_f1 = staticmethod(weighted_f1)

```

5. Experiment

Eight combinations compare two learning rates and four L2 strengths. Candidates are trained on an inner fit split and selected by validation macro F1; only the winner is refitted on all outer-training rows and evaluated once on the test set. This avoids selecting on the test set.

```

In [19]: results = json.loads((ROOT / 'figures' / 'experiment_results.json').read_text())
runs = pd.DataFrame(results['all_runs']).sort_values('validation_macro_f1',
runs

```

```

Out[19]:

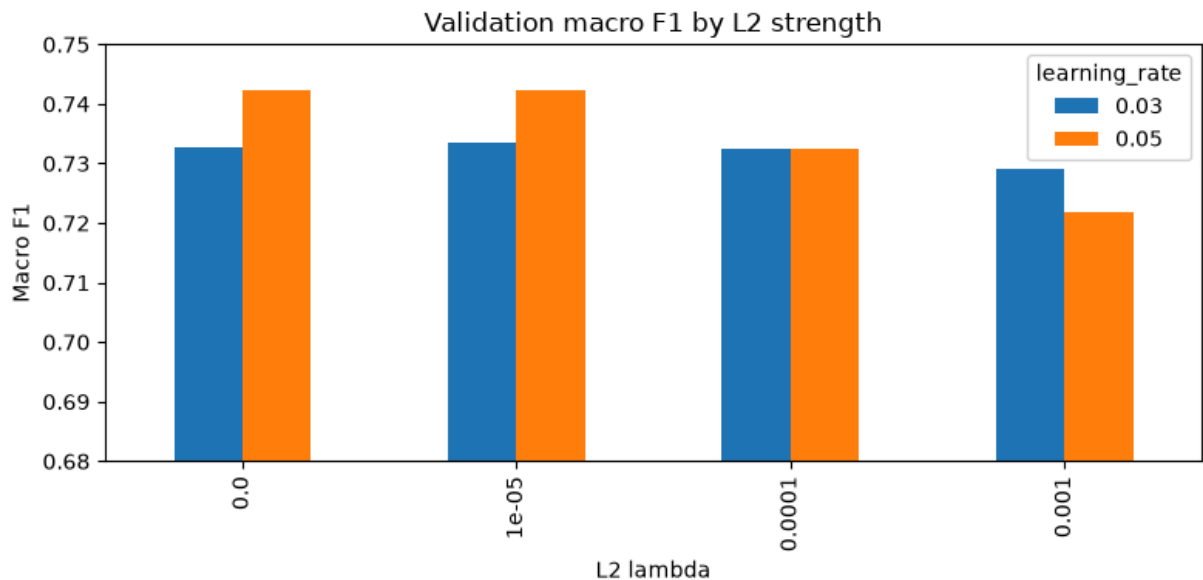
```

	l2_lambda	learning_rate	validation_accuracy	validation_macro_f1	validation_weighted_f1
1	0.00000	0.05	0.748393	0.742420	0.742420
3	0.00001	0.05	0.748393	0.742420	0.742420
2	0.00001	0.03	0.740129	0.733579	0.733579
0	0.00000	0.03	0.739210	0.732685	0.732685
4	0.00010	0.03	0.739210	0.732533	0.732533
5	0.00010	0.05	0.739210	0.732517	0.732517
6	0.00100	0.03	0.735537	0.729050	0.729050
7	0.00100	0.05	0.730946	0.721956	0.721956

```

In [20]: ax = runs.pivot(index='l2_lambda', columns='learning_rate', values='validation_macro_f1')
ax.set(title='Validation macro F1 by L2 strength', ylabel='Macro F1', xlabel='L2 strength')
plt.tight_layout()
plt.savefig(ROOT / 'figures' / 'a3_experiment_comparison.png', dpi=160)
plt.show()

```



```
In [21]: model = joblib.load(ROOT / 'app' / 'models' / 'car_price_classifier.joblib')
pd.DataFrame(model.test_report_).T
```

```
Out[21]:
```

	precision	recall	f1-score	support
0	0.846154	0.857527	0.851802	372.000000
1	0.637821	0.614198	0.625786	324.000000
2	0.623457	0.601190	0.612121	336.000000
3	0.816092	0.863222	0.838996	329.000000
accuracy	0.737693	0.737693	0.737693	0.737693
macro avg	0.730881	0.734034	0.732176	1361.000000
weighted avg	0.734312	0.737693	0.735729	1361.000000

The selected unregularised model (`lambda=0` , learning rate `0.05`) achieved **0.742 validation macro F1** and, after a full training refit, **0.738 test accuracy** and **0.732 test macro F1**. Small L2 values tied or nearly tied the winner; stronger L2 slightly underfit. The scratch and scikit-learn report values match numerically.

6. MLflow model registry and deployment

MLflow may use a configured remote server or a local server. The app tries the remote registry first and falls back to its local registry when it is unavailable. Training runs are logged to the selected URI; local and remote histories are separate.

`training/train.py` defaults to tracking URI `http://127.0.0.1:5000` , experiment `st127314-a3` , logs parameters/metrics without logging the dataset, saves the final pipeline, and can register `st127314-a3-model` at Staging. Run:

```
python training/train.py --log-mlflow --register
```

The repository also contains the Dash application, Docker image, two model-interface unit tests, and a GitHub Actions workflow. Every push runs tests; a passing push builds and publishes the Docker image. Docker Hub credentials are stored as GitHub secrets, never in the repository.

MLflow UI screenshots

The runs above are visible in the MLflow tracking UI, and the best refit is registered in the Model Registry at the Staging stage.

Experiment runs (st127314-a3):

mlflow3.16.0

GenAIModel training

← st127314-a3

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Review

Datasets

Evaluation runs

Prompts & versions

▶ Playground

Prompts

Agent versions

AI GatewayNew

Docs

Settings

Experiments > st127314-a3 >

Training runs

Views

metrics.rmse < 1 and params.model = "tree"

+ New run

All timeState: ActiveDatasetsSort: CreatedColumns

Group by

	Run Name	Created	Dataset	Duration
☐	● best-model-refit	✓ 38 minutes ago	-	1.7s
☐	● l2=0.001-lr=0.05	✓ 38 minutes ago	-	254ms
☐	● l2=0.001-lr=0.03	✓ 38 minutes ago	-	338ms
☐	● l2=0.0001-lr=0.05	✓ 38 minutes ago	-	340ms
☐	● l2=0.0001-lr=0.03	✓ 38 minutes ago	-	412ms
☐	● l2=1e-05-lr=0.05	✓ 38 minutes ago	-	414ms
☐	● l2=1e-05-lr=0.03	✓ 38 minutes ago	-	410ms
☐	● l2=0.0-lr=0.05	✓ 38 minutes ago	-	414ms
☐	● l2=0.0-lr=0.03	✓ 38 minutes ago	-	429ms

9 matching runs

Best model run (best-model-refit) with logged test metrics:

mlflow3.16.0

GenAIModel training

← st127314-a3

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Review

Datasets

Evaluation runs

Prompts & versions

▶ Playground

Prompts

Agent versions

AI GatewayNew

Docs

Settings

st127314-a3 > Runs >

best-model-refit

OverviewModel metricsSystem metricsTracesArtifacts

About this run

Created at

09/30/2026, 07:14:00 AM

Created by

kamalhosen

Experiment ID

1

Status

Finished

Run ID

d4888f5f6d904368bc5ef21ec13d7130

Duration

1.7s

Source

train.pymain7cb2826

Registered prompts

—

Datasets

None

Tags

Add tags

Registered models

st127314-a3-modelv1

Description

No description

Metrics (3)

Search metrics

Metric	Value	Models
test_accuracy	0.7376928728875827	model
test_macro_f1	0.7321763368832184	model
test_weighted_f1	0.7357292287607183	model

Registered model (st127314-a3-model , Version 1, Stage: Staging):

mlflow

3.16.0

GenAI

Model training

Home

Experiments

Prompts

MCP registryBeta

AI GatewayNew

Docs

Settings

Registered Models > st127314-a3-model >

Version 1

Promote model

Registered At: 09/30/2026 07:14:01 AM

Last Modified: 09/30/2026 07:14:01 AM

Source Run: best-model-refit

Aliases: Add

Stage (deprecated): Staging

New model registry UI

> Description

Edit

> Tags

> Schema

Name	Type
+ Inputs (0)	
+ Outputs (0)	